

6. Dynamic Programming

Contents

LIS	1
0/1 Knapsack	2
Edit Distance	2
Longest Common Subsequence	3
Shortest Common Supersequence	3
Count Subsets Sum to K	4
Hamiltonian Paths	4
Kadane	4
Dynamic max subarray sum	5
Meet in the middle	5
Digit DP	6

LIS

```
1 int lis_size(const vector<int>& nums) {
2     vector<int> tail;
3     for (auto x : nums) {
4         auto it = lower_bound(tail.begin(), tail.end(), x);
5         if (it == tail.end()) tail.push_back(x);
6         else *it = x;
7     }
8     return tail.size();
9 }
10
11 int lis_with_sequence(const vector<int>& a,
12                      vector<int>& out_seq)
13 {
14     int n = a.size();
15     vector<int> tail;
16     vector<int> tail_idx;
17     vector<int> prev(n, -1);
18     tail.reserve(n);
19     tail_idx.reserve(n);
20
21     for (int i = 0; i < n; ++i) {
22         int x = a[i];
23         auto it = lower_bound(tail.begin(), tail.end(), x);
24         int pos = it - tail.begin();
25         if (it == tail.end()) {
26             tail.push_back(x);
27             tail_idx.push_back(i);
28         }
29         else {
30             *it = x;
31             tail_idx[pos] = i;
```

```

32         }
33         if (pos > 0)
34             prev[i] = tail_idx[pos-1];
35     }
36
37     int lis_len = tail.size();
38     out_seq.clear();
39     for (int cur = tail_idx[lis_len - 1]; cur >= 0; cur = prev[cur])
40         out_seq.push_back(a[cur]);
41     reverse(out_seq.begin(), out_seq.end());
42
43     return lis_len;
44 }

```

0/1 Knapsack

```

1  int knapsack(int W, vector<int> &val, vector<int> &wt) {
2      vector<int> dp(W + 1, 0);
3      for (int i = 1; i <= wt.size(); i++)
4          for (int j = W; j >= wt[i - 1]; j--)
5              dp[j] = max(dp[j], dp[j - wt[i - 1]] + val[i - 1]);
6
7
8      return dp[W];
9  }
10
11 int unbounded_knapSack(int capacity, vector<int> &val, vector<int> &wt) {
12     vector<int> dp(capacity + 1, 0);
13     for (int i = val.size() - 1; i >= 0; i--) {
14         for (int j = 1; j <= capacity; j++) {
15             int take = 0;
16             if (j - wt[i] >= 0) {
17                 take = val[i] + dp[j - wt[i]];
18             }
19             int noTake = dp[j];
20             dp[j] = max(take, noTake);
21         }
22     }
23     return dp[capacity];
24 }

```

Edit Distance

```

1  string a, b; cin >> a >> b;
2  int na = a.size(), nb = b.size();
3  int dp[na+1][nb+1];
4  memset(dp, 0, sizeof dp);
5
6  for (int i = 1; i <= na; i++)
7      dp[i][0] = i;
8  for (int j = 1; j <= nb; j++)
9      dp[0][j] = j;
10 for (int i = 1; i <= na; i++) {
11     for (int j = 1; j <= nb; j++) {
12         dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;

```

```

13         if (a[i-1] == b[j-1])
14             dp[i][j] = min(dp[i][j], dp[i-1][j-1]);
15     }
16 }
17 cout << dp[na][nb];

```

Longest Common Subsequence

```

1 void lcs(char* X, char* Y, int m, int n)
2 {
3     int L[m + 1][n + 1];
4     for (int i = 0; i <= m; i++) {
5         for (int j = 0; j <= n; j++) {
6             if (i == 0 || j == 0)
7                 L[i][j] = 0;
8             else if (X[i - 1] == Y[j - 1])
9                 L[i][j] = L[i - 1][j - 1] + 1;
10            else
11                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
12        }
13    }
14    int index = L[m][n];
15
16    char lcs[index + 1];
17    lcs[index] = '\0';
18    int i = m, j = n;
19    while (i > 0 && j > 0) {
20        if (X[i - 1] == Y[j - 1]) {
21            lcs[index - 1]
22                = X[i - 1];
23            i--;
24            j--;
25            index--;
26        }
27        else if (L[i - 1][j] > L[i][j - 1])
28            i--;
29        else
30            j--;
31    }
32 }

```

Shortest Common Supersequence

```

1 while (i > 0 && j > 0) {
2     if (str1[i - 1] == str2[j - 1]) {
3         result.push_back(str1[i - 1]);
4         i--;
5         j--;
6     } else if (dp[i - 1][j] > dp[i][j - 1]) {
7         result.push_back(str1[i - 1]);
8         i--;
9     } else {
10        result.push_back(str2[j - 1]);
11        j--;
12    }
13 }

```

```

14 while (i > 0) {
15     result.push_back(str1[i - 1]);
16     i--;
17 }
18 while (j > 0) {
19     result.push_back(str2[j - 1]);
20     j--;
21 }
22 reverse(result.begin(), result.end());
23 return result;

```

Count Subsets Sum to K

```

1 int perfectSum(vector<int> &arr, int target) {
2     int n = arr.size();
3     vector<int> dp(target + 1, 0), ndp(target + 1, 0);
4     dp[0] = 1;
5     for (int i = 1; i <= n; i++) {
6         ndp = dp;
7         for (int j = 0; j <= target; j++) {
8             if (j >= arr[i - 1]) {
9                 ndp[j] += dp[j - arr[i - 1]];
10            }
11        }
12        dp = ndp;
13    }
14    return ndp[target];
15 }

```

Hamiltonian Paths

```

1 int rec(int u, int vis) {
2     vis |= (1 << u);
3     if (u == n-1)
4         return __builtin_popcount(vis) == n;
5
6     int& ans = dp[vis][u];
7     if (~ans) return ans;
8     // dpid[u][vis] = 2;
9     ans = 0;
10    each(ver, g[u])
11        if (!(vis >> ver) & 1)
12            ans = (ans + rec(ver, vis | (1 << ver)))%mod;
13    return ans;
14 }

```

Kadane

```

1 template<class T>
2 array<ll, 3> kadane(const vector<T>& arr) {
3     ll mx = LLONG_MIN, csum = 0;
4     ll s = 0, e = 0, ts = 0, sz = arr.size();
5     for (ll i = 0; i < sz; i++) {
6         if (csum + arr[i] > arr[i]) {
7             csum += arr[i];

```

```

8         } else {
9             csum = arr[i];
10            ts = i;
11        }
12        if (csum > mx) {
13            mx = csum;
14            s = ts;
15            e = i;
16        }
17    }
18    return {mx, s, e};
19 }

```

Dynamic max subarray sum

```

1 struct info {
2     int sum, pref, suff, ans, mnelement = -1e4-10;;
3     info(int x) {
4         sum = pref = suff = x;
5         // ans = max<int>(x, 0);
6         ans = x; // if empty subarray is not allowed
7     }
8     info() { // default value
9         sum = pref = suff = ans = mnelement;
10    }
11    friend info operator+(const info &l, const info &r) {
12        info ret;
13        ret.sum = l.sum + r.sum;
14        ret.pref = max(l.pref, l.sum + r.pref);
15        ret.suff = max(r.suff, r.sum + l.suff);
16        ret.ans = max({l.ans, r.ans, l.suff + r.pref});
17        return ret;
18    }
19 };

```

Meet in the middle

```

1 vector<int> a;
2 auto get_subset_sums = [&](int l, int r) -> vector<ll> {
3     int len = r - l + 1;
4     vector<ll> res;
5     for (int i = 0; i < (1 << len); i++) {
6         ll sum = 0;
7         for (int j = 0; j < len; j++)
8             if (i & (1 << j))
9                 sum += a[l + j];
10
11        res.push_back(sum);
12    }
13    return res;
14 };
15 vector<ll> left = get_subset_sums(0, n / 2 - 1);
16 vector<ll> right = get_subset_sums(n / 2, n - 1);
17 sort(left.begin(), left.end());
18 sort(right.begin(), right.end());
19 ll ans = 0;

```

```

20 for (ll i : left) {
21     auto low_iterator = lower_bound(right.begin(), right.end(), x - i);
22     auto high_iterator = upper_bound(right.begin(), right.end(), x - i);
23     ans += high_iterator - low_iterator;
24 }

```

Digit DP

```

1 using ull = unsigned long long int;
2 #define int ull
3 pair<int, int> dp[2][16];
4 bool vis[2][16];
5 string num;
6 pair<int, int> rec(int idx, bool U) {
7     if (idx == num.size()) return {0, 1};
8     pair<int, int>& ret = dp[U][idx];
9     if (vis[U][idx]) return ret;
10    vis[U][idx] = 1;
11    ret = {0, 0};
12    int D = U ? num[idx] - '0' : 9;
13    for (int d = 0; d <= D; d++) {
14        auto [x, y] = rec(idx+1, U&d==D);
15        ret.first += x + d * y;
16        ret.second += y;
17    }
18    return ret;
19 }

```